

Andrei Nekrashevich <axolm13@gmail.com>, <axolm@yandex-team.ru>
Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2021-05-25

Stacktrace from exception

I. Introduction and Motivation

Quite often Standard Library exceptions do not provide enough information to diagnose the error. Consider the example:

```
#include <iostream>
#include <stdexcept>
#include <string_view>

void foo(std::string_view key);
void bar(std::string_view key);

int main() {
    try {
        foo("test1");
        bar("test2");
    } catch (const std::exception& exc) {
        std::cerr << "Caught exception: " << exc.what() << '\n';
    }
}
```

The output of the above sample may be the following:

Caught exception: map::at

That output is quite useless because it does not help the developer to understand in what function the error happened.

This paper proposes to add an ability to get stacktrace of a caught exception, namely to add static method `std::stacktrace::from_current_exception()`:

```
#include <iostream>
#include <stdexcept>
#include <string_view>
#include <stacktrace> // <---

void foo(std::string_view key);
void bar(std::string_view key);

int main() {
    try {
        foo("test1");
        bar("test2");
    } catch (const std::exception& exc) {
        std::stacktrace trace = std::stacktrace::from_current_exception(); // <---
        std::cerr << "Caught exception: " << exc.what() << ", trace:\n" << trace;
    }
}
```

The output of the above sample may be the following:

Caught exception: map::at, trace:
0# get_data_from_config(std::string_view) at /home/axolm/basic.cpp:600
1# bar(std::string_view) at /home/axolm/basic.cpp:6
2# main at /home/axolm/basic.cpp:17

That output is quite useful! Without a debugger we can locate the source file and the function that has thrown the exception.

More production log examples where stacktraces would help to diagnose the problem faster:

- Serializing 'UserData' failed: bad optional access — a trace would show that the exception is from NormalizePhone function; further logs analysis will show that the phone is missing.
- ERROR [/order-search] exception in 'handler-search-cab-orders' handler in handle_request: _Map_base::at (std::out_of_range) — a trace would show that the exception is from GetOrganization function; reading the code would show that the function has issues with concurrent updates
- Failed to process 'POST' request to '/v1/order-a-cab': bad any_cast — a trace would show that the exception is from authorization function; wrong context is passed to the authorization function.
- Fatal: Missing a colon after a name of object member . — a trace would show that the exception is from a function that parses JSON dictionary of configuration variables.

Here's another motivation example. It allows the developer to diagnose `std::terminate` calls because of throws in `noexcept` function:

```
#include <iostream>
#include <stacktrace>
#include <stdexcept>
#include <unordered_map>

void broken_function() noexcept {
    std::unordered_map<std::string, int> m;
    [[maybe_unused]] auto value = m.at("non-existing-key");
}

int main() {
    std::set_terminate([] {
        auto trace = std::stacktrace::from_current_exception();
        if (trace) {
            std::cerr << "Terminate was called with an active exception:\n"
                        << trace << std::endl;
        }
    });

    broken_function();
}
```

Output:

Exception trace:

```
0# std::__throw_out_of_range(char const*) at /build/gcc/src/gcc/libstdc++-v3/src/c++11/functexcept.cc:82
1# std::__detail::_Map_base<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator ...
2# std::unordered_map<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > ...
3# broken_function() at /home/axolm/terminate.cpp:8
4# main at /home/axolm/terminate.cpp:17
```

Disclaimer: may not produce traces on all platforms

Finally, an ability to extract stacktraces from exceptions addresses feature requests from "2021 Annual C++ Developer Survey". In it some people were complaining that in their projects exceptions are restricted due to lack of stacktraces.

II. Implementation and ABI stability

Majority of the popular platforms have special functions to allocate/throw/deallocate exceptions: `__cxa_allocate_exception`, `__cxa_throw`, `__CxxThrowException`... To embed a trace into an exception an underlying implementation may be changed, to gather the trace at the point where the exception is thrown.

This can be done without breaking the ABI. Here's a prototype [libsfe](#) that replaces those functions and adds a stacktrace into each exception. Application can switch between exceptions with traces and without via adding and removing a

`LD_PRELOAD=/usr/local/lib/libsfe_preload.so`.

III. Use cases

The table bellow shows `std::stacktrace` related techniques to use in different cases

Case	HowTo	Proposal
Need a way to get diagnostic info for an unrecoverable errors. For example for failed assert checks	Just log the trace and abort: <pre>std::cerr << std::stacktrace::current(); std::abort();</pre>	P0881
Need to get stacktraces from some code paths and you have control over the throw site.	Manually embed the stacktrace into the exception: <pre>template <class Base> struct TracedException: Base, std::stacktrace { TracedException(): Base(), std::stacktrace(std::stacktrace::current()) {} }; // ... throw TracedException<std::bad_any_cast>();</pre> Extract it at the catch site: <pre>catch (const std::exception& e) { if (auto ptr = dynamic_cast<const std::stacktrace*>(&e); ptr) { std::cerr << *ptr; } // ... }</pre>	P0881
Need a trace from the throw site to simplify diagnoses of failed tests.	Use <code>stacktrace::from_current_exception</code> in your testing macro to get the trace: <pre>EXPECT_NO_THROW(search_engine_impl("42"));</pre>	This proposal
Need a way to diagnose a running application in production.	Turn on <code>stacktrace::from_current_exception</code> in your error handling logic: <pre>try { std::this_thread::capture_stacktraces_at_throw(trace_exceptions_var); process_request(request); } catch (const std::exception& e) { if (auto trace = std::stacktrace::from_current_exception(); trace) LOG_PROD_DEBUG() << e.what() << " at " << trace; }</pre>	This proposal

To sum up: you need functionality from this paper when you have no control over the throw site or changing all the throw sites is time consuming.

IV. Design decisions

A. Recommend to turn off by default

As was noted in mailing list discussion the proposed functionality increases memory consumption by exceptions. Moreover, it adds noticeable overhead on some platforms.

Because of that we recommend to disable the functionality by default in release builds and enable in debug. To enable the stacktrace capturing on exception object construction for the current thread user would have to use `std::this_thread::capture_stacktraces_at_throw(bool enable) noexcept;`.

B. Safety

Proposed `std::stacktrace::from_current_exception()` function is a diagnostic facility. It would be used in error processing and other places that are rarely tested. Because of that it follows the `std::stacktrace` design: do not report missing traces as errors, do not throw and treat all the internal errors as a missing trace.

C. Link time flag from `std::stacktrace`

"A Proposal to add `stacktrace` library" [P0881](#) encouraged implementations to provide a link time flag to disable or enable traces. With disabled tracing `std::stacktrace::from_current_exception()` may be called at runtime and it returns default constructed `std::stacktrace`.

D. Per thread ability to enable

Consider some server under heavy load with 5 different executors. Something goes wrong with one of the executors. Capturing stacktraces on exception object construction and symbolizing them for all the executors could slow down the server significantly while only traces for 1 executor are needed. So the option should be thread specific.

Such approach scales well to coroutines. Call the `std::this_thread::capture_stacktraces_at_throw`(`bool` enable) `noexcept`; after context switch and you have per coroutine ability to enable or disable stacktrace capturing at the exception object construction.

V. Wording

Add to the [stacktrace.syn]:

```
// [stacktrace.basic], class template basic_stacktrace
template<class Allocator>
class basic_stacktrace;

namespace this_thread {
    void capture_stacktraces_at_throw(bool enable = true) noexcept;
}

// basic_stacktrace typedef names
using stacktrace = basic_stacktrace<allocator<stacktrace_entry>;>;
```

Add to the [stacktrace.basic.overview]:

```
static basic_stacktrace current(size_type skip, size_type max_depth,
                                const allocator_type& alloc = allocator_type()) noexcept;

static basic_stacktrace from_current_exception(
    const allocator_type& alloc = allocator_type()) noexcept;

basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
```

Add to the [stacktrace.basic.ctor] after the description of `current()` functions:

```
static basic_stacktrace from_current_exception(const allocator_type& alloc = allocator_type()) noexcept;
```

Effects: Returns a `basic_stacktrace` object with `frames_` storing the stacktrace of the evaluation at the point of construction of the currently handled exception, or an empty `basic_stacktrace` object if the initialization of `frames_` failed, stacktrace captures were not enabled for the throwing thread or no exception is being handled. `alloc` is passed to the constructor of the `frames_` object.

[Note: Rethrowing exceptions with `throw`; do not change stacktrace, throwing exception with `throw e`; stores a new stacktrace.- end note]

Add section [stacktrace.thread.this] before the [stacktrace.basic.mod] section:

Namespace `this_thread`

[stacktrace.thread.this]

```
void this_thread::capture_stacktraces_at_throw(bool enable = true) noexcept;
```

Effects: Invoking the function with the `enable` parameter equal to `true` enables capturing stacktraces by the current thread of execution at exception object construction, disables otherwise.

Recommended practice: Implementations should disable the capturing stacktraces by the current thread of execution at exception object construction in release builds by default and enable in debug builds.

VI. Revision History

Revision 0:

- Initial proposal.

VII. References

[libsfe] Proof of concept implementation. Available online at <https://github.com/axolm/libsfe>.